

Tarea 2

501404, 503309 — Análisis de Algoritmos

Erick Saldivar

Alejandro Neira

Fecha de entrega: lunes 6 de julio de 2026

1. Diseño

Notación. $G = (V, E)$ es un grafo dirigido con pesos $w: E \rightarrow \mathbb{R}$, $n := |V|$ y $m := |E|$. Un camino de u a v es una secuencia $p = \langle v_0, \dots, v_\ell \rangle$ con $v_0 = u$, $v_\ell = v$ y $(v_{i-1}, v_i) \in E$; su peso es $w(p) = \sum_{i=1}^{\ell} w(v_{i-1}, v_i)$, es *simple* si no repite vértices y sus *vértices intermedios* son $v_1, \dots, v_{\ell-1}$. La distancia mínima es $\delta(u, v) := \inf\{w(p) : p \text{ camino de } u \text{ a } v\}$, con valor $+\infty$ si no existe camino, y el problema APSP consiste en calcular $\delta(u, v)$ para todo par $(u, v) \in V \times V$. Si G contiene un ciclo de peso negativo, las distancias a través de él no están bien definidas (recorriéndolo el peso decrece sin cota); suponemos que no los hay, y ambos algoritmos detectan y reportan esta situación cuando ocurre.

1.1. Algoritmo Base: APSP mediante n llamadas a Bellman-Ford

El algoritmo de Bellman-Ford, visto en clases, resuelve el problema del camino más corto con una fuente (SSSP): dado $s \in V$, calcula $\delta(s, v)$ para todos los $v \in V$ a la vez, relajando las m aristas durante $n - 1$ rondas, y con una ronda adicional detecta si existe algún ciclo negativo alcanzable desde s [1, cap. 24]. Las $n - 1$ rondas bastan porque, sin ciclos negativos, los caminos más cortos pueden tomarse *simples* (a lo más $n - 1$ aristas): eliminar de un camino un ciclo, de peso ≥ 0 , no aumenta su peso.

Argumento. La solución de APSP es la matriz D de $n \times n$ definida por $D[u][v] = \delta(u, v)$. Su fila u , es decir la familia de valores $(\delta(u, v))_{v \in V}$, es *exactamente* la salida del problema SSSP con fuente u , que una llamada $\text{BELLMANFORD}(G, w, u)$ calcula correctamente. Como las n filas cubren todos los pares de $V \times V$, basta ejecutar Bellman-Ford una vez por cada $u \in V$ para obtener la matriz completa: n llamadas resuelven APSP. El procedimiento resultante es el *Algoritmo Base* (Algoritmo 1). Nótese además que todo ciclo negativo es alcanzable desde sus propios vértices y que el Algoritmo Base ejecuta Bellman-Ford desde *todas* las fuentes, por lo que detecta cualquier ciclo negativo de G .

Algoritmo 1 ALGORITMOBASE(G, w)

Entrada: grafo dirigido $G = (V, E)$ con pesos w

Salida: matriz D con $D[u][v] = \delta(u, v)$ para todo $(u, v) \in V \times V$

- 1: **para** cada $u \in V$ **hacer**
 - 2: $D[u][\cdot] \leftarrow \text{BELLMANFORD}(G, w, u)$
 - 3: **fin para**
 - 4: **retornar** D
-

Análisis de complejidad. Cada llamada a Bellman-Ford ejecuta a lo más n rondas ($n - 1$ de relajación más la de detección), cada una con trabajo $O(1)$ por arista; junto con la inicialización, una llamada cuesta $O(n + nm) = O(nm)$. El Algoritmo Base realiza n llamadas, para un total de

$$O(n^2m).$$

En grafos *densos* ($m = \Theta(n^2)$) esto es $O(n^4)$; en grafos *dispersos* ($m = O(n)$), $O(n^3)$. El espacio es $\Theta(n^2)$: el tamaño de la propia salida, más $O(n)$ auxiliar por llamada (el vector de distancias).

1.2. Algoritmo de Floyd-Warshall

Diseñamos un algoritmo de programación dinámica que resuelve APSP en tiempo $\Theta(n^3)$, sin depender de m . Identificamos los vértices con los enteros $V = \{1, 2, \dots, n\}$.

Subproblemas. La dificultad de APSP está en que un camino más corto de i a j puede pasar por cualquier subconjunto de vértices. La idea es *restringir qué vértices pueden usarse como intermedios* e ir liberando la restricción gradualmente: para $0 \leq k \leq n$, sea $V_k := \{1, \dots, k\}$ (con $V_0 = \emptyset$) y

$$\delta^{(k)}(i, j) := \inf\{w(p) : p \text{ es un camino de } i \text{ a } j \text{ con todos sus vértices intermedios en } V_k\},$$

con valor $+\infty$ si no existe tal camino; los *extremos* i, j no están restringidos. Como $V_n = V$ no impone restricción alguna, $\delta^{(n)} = \delta$: resolver los subproblemas de nivel n resuelve APSP. Notemos que, sin ciclos negativos, todo camino se transforma en uno *simple* de peso menor o igual eliminando ciclos (cada uno de peso ≥ 0) y sin agregar intermedios nuevos; el ínfimo se alcanza entonces en un camino simple, y en particular $\delta^{(k)}(u, u) = 0$ para todo u, k .

Recurrencia. Sea $k \geq 1$ y sea p un camino más corto de i a j con intermedios en V_k , tomado simple. Hay exactamente dos casos:

- k no es intermedio de p : todos los intermedios de p están en V_{k-1} , y p es candidato del subproblema de nivel $k - 1$;
- k es intermedio de p : como p es simple, k aparece exactamente una vez y descompone a p en $p_1: i \rightarrow k$ y $p_2: k \rightarrow j$, ambos con intermedios en V_{k-1} . Por la subestructura óptima de los caminos más cortos (vista en clases: todo subcamino de un camino más corto es un camino más corto entre sus extremos, pues si hubiera uno mejor, sustituirlo daría un camino total mejor), p_1 y p_2 son óptimos en sus subproblemas de nivel $k - 1$.

Esto motiva definir los valores de programación dinámica

$$D_{ij}^{(0)} = \begin{cases} 0 & \text{si } i = j, \\ w(i, j) & \text{si } i \neq j \text{ y } (i, j) \in E, \\ +\infty & \text{en otro caso,} \end{cases} \quad D_{ij}^{(k)} = \min\left(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)}\right) \quad (1)$$

para $1 \leq k \leq n$. (Con aristas paralelas (i, j) , en $D_{ij}^{(0)}$ se toma la de menor peso.)

Correctitud. Probamos por inducción sobre k que $D_{ij}^{(k)} = \delta^{(k)}(i, j)$ para todo par (i, j) ; con $k = n$ esto da $D_{ij}^{(n)} = \delta(i, j)$. Caso base $k = 0$: un camino sin intermedios es el camino trivial $\langle i \rangle$ (solo si $i = j$, de peso 0; un lazo (i, i) de peso menor sería un ciclo negativo) o una única arista (i, j) , y $D_{ij}^{(0)}$ toma el mínimo entre estas posibilidades. Paso inductivo: supongamos $D^{(k-1)} = \delta^{(k-1)}$. ($\geq$) Cada argumento finito del mínimo en (1) es el peso de un camino real de i a j con intermedios en V_k : el primero, de un camino con intermedios en $V_{k-1} \subseteq V_k$; el segundo, de la concatenación de caminos

óptimos $i \rightarrow k$ y $k \rightarrow j$ con intermedios en V_{k-1} (la concatenación puede repetir vértices, pero sigue siendo un camino). Luego $D_{ij}^{(k)} \geq \delta^{(k)}(i, j)$. ($\leq$) Sea p un camino más corto (simple) de i a j con intermedios en V_k ; por el análisis de casos de la recurrencia, si k no es intermedio de p entonces $D_{ij}^{(k)} \leq D_{ij}^{(k-1)} = \delta^{(k-1)}(i, j) \leq w(p)$, y si k es intermedio entonces $D_{ij}^{(k)} \leq D_{ik}^{(k-1)} + D_{kj}^{(k-1)} = w(p_1) + w(p_2) = w(p)$. En ambos casos $D_{ij}^{(k)} \leq \delta^{(k)}(i, j)$, lo que completa la inducción.

Algoritmo. La recurrencia puede evaluarse sobre una *única* matriz D actualizada in situ. En efecto, como $D_{kk}^{(k-1)} = \delta^{(k-1)}(k, k) = 0$, la etapa k deja invariantes la fila y la columna k : $D_{ik}^{(k)} = \min(D_{ik}^{(k-1)}, D_{ik}^{(k-1)} + 0) = D_{ik}^{(k-1)}$ y simétricamente $D_{kj}^{(k)} = D_{kj}^{(k-1)}$; y esas son exactamente las entradas que la etapa k lee. El resultado es el algoritmo de Floyd-Warshall (Algoritmo 2).

Algoritmo 2 FLOYDWARSHALL(G, w)

Entrada: grafo dirigido $G = (V, E)$ con pesos w , $V = \{1, \dots, n\}$

Salida: matriz D con $D[i][j] = \delta(i, j)$, o reporte de ciclo negativo

```

1:  $D[i][j] \leftarrow 0$  si  $i = j$ ;  $w(i, j)$  si  $(i, j) \in E$ ;  $\infty$  en otro caso  $\triangleright D = D^{(0)}$ 
2: para  $k \leftarrow 1$  hasta  $n$  hacer
3:   para  $i \leftarrow 1$  hasta  $n$  hacer
4:     para  $j \leftarrow 1$  hasta  $n$  hacer
5:       si  $D[i][k] + D[k][j] < D[i][j]$  entonces
6:          $D[i][j] \leftarrow D[i][k] + D[k][j]$ 
7:       fin si
8:     fin para
9:   fin para
10: fin para
11: si  $D[i][i] < 0$  para algún  $i \in V$  entonces
12:   reportar ciclo negativo
13: fin si
14: retornar  $D$ 
```

Detección de ciclos negativos. La comprobación final del Algoritmo 2 es correcta: al terminar existe $D[i][i] < 0$ si y solo si G contiene un ciclo negativo. ($\Rightarrow$) Toda entrada finita de D es en todo momento el peso de un camino real (vale al inicializar, y cada actualización concatena dos caminos reales $i \rightarrow k$ y $k \rightarrow j$); si $D[i][i] < 0$ existe un camino cerrado de peso negativo, y como todo camino cerrado se descompone en ciclos simples, alguno de ellos es negativo. ($\Leftarrow$) La desigualdad ($\leq$) de la correctitud vale *sin hipótesis sobre ciclos*: tras la etapa k , $D[i][j] \leq w(p)$ para todo camino simple p de i a j con intermedios en V_k (misma inducción, usando que las entradas nunca aumentan). Sea C un ciclo simple de peso negativo. Si es un lazo (u, u) , la inicialización ya deja $D[u][u] \leq w(u, u) < 0$. Si no, sean x e y sus dos vértices de mayor índice: y parte a C en dos caminos simples $p_1: x \rightarrow y$ y $p_2: y \rightarrow x$ con intermedios en V_{y-1} , y al procesar el par (x, x) en la etapa $k = y$ queda $D[x][x] \leq D[x][y] + D[y][x] \leq w(p_1) + w(p_2) = w(C) < 0$, cota que se conserva hasta el final porque las entradas solo decrecen.

Análisis de complejidad. La inicialización cuesta $\Theta(n^2)$ más $O(m)$ por recorrer las aristas. El cuerpo del triple bucle realiza trabajo $O(1)$ (una suma, una comparación y a lo más una asignación) y se ejecuta exactamente n^3 veces; la revisión final de la diagonal cuesta $O(n)$. El total es

$$\Theta(n^3),$$

independiente de m , con espacio $\Theta(n^2)$: una única matriz, que coincide con el tamaño de la salida. Frente al Algoritmo Base ($O(n^2m)$), en grafos densos Floyd-Warshall gana un factor n completo

($\Theta(n^3)$ contra $O(n^4)$); en grafos dispersos ambos son $O(n^3)$, pero el bucle interno de Floyd-Warshall solo hace aritmética sobre una matriz contigua en memoria, con constantes menores. La comparación empírica es objeto de la sección de experimentos.

2. Implementación y verificación de correctitud (pregunta 2.1)

2.1. Entorno y decisiones de implementación

Ambos algoritmos fueron implementados en **C++17**, sin emplear librerías con subrutinas para grafos: solo se usan contenedores estándar (`std::vector`) y utilidades de entrada/salida. El código se compila con **g++** (GCC 16.1.0, MinGW-w64 de MSYS2) usando la opción `-O2`. Las decisiones principales son las siguientes.

- **Representación del grafo** (`src/grafos.hpp`): lista de aristas **Arista** = (u, v, w) con nodos 0-indexados y pesos **double**. Bellman-Ford recorre precisamente la lista de aristas, y Floyd-Warshall solo la usa para construir $D^{(0)}$, por lo que esta representación sirve a ambos sin estructuras adicionales.
- **Infinito**: la ausencia de camino se representa con `INF = std::numeric_limits<double>::infinity()`. La aritmética IEEE 754 garantiza $\infty + c = \infty$ e $\infty + \infty = \infty$, y que toda comparación $\infty + c < x$ con x finito sea falsa; así las relajaciones no requieren casos especiales.
- **Bellman-Ford** (`bellman_ford` en `src/apsp.hpp`): implementa el algoritmo visto en clases (sección 1.1) con una mejora estándar: si una ronda completa no produce ninguna relajación, el vector de distancias alcanzó un punto fijo y el algoritmo termina antes. Esto no afecta la correctitud (el punto fijo ya es la solución) pero reduce el tiempo en instancias favorables. La ronda extra de detección de ciclos negativos se realiza siempre.
- **Algoritmo Base** (`algoritmo_base`): n llamadas a `bellman_ford`, una por fuente, exactamente como en el Algoritmo 1.
- **Floyd-Warshall** (`floyd_warshall`): transcripción directa del Algoritmo 2, con la actualización in situ justificada en la sección 1.2. Si la entrada contiene aristas paralelas, en $D^{(0)}$ se conserva la de menor peso; los lazos de peso negativo quedan en la diagonal y son detectados como ciclos negativos.
- **Detección de ciclos negativos**: ambas funciones aceptan un puntero opcional `ciclo_negativo` donde reportan la detección justificada en las secciones 1.1 y 1.2, lo que permite contrastar también este aspecto entre ambos algoritmos.

Las instancias se leen desde archivos de texto con el formato siguiente (nodos 0-indexados; una arista dirigida por línea):

```
n m
u v w          (m líneas: arista u -> v con peso real w)
```

2.2. Marco comparativo para verificar la correctitud

La verificación está automatizada en el programa `src/verificar.cpp`, que ejecuta dos baterías de pruebas y termina con código de salida 0 solo si todas pasan.

Batería 1: instancias con solución conocida. Cuatro instancias pequeñas cuya matriz de distancias completa se conoce de antemano (calculada a mano o tomada de la literatura), elegidas para cubrir modos de falla distintos. Cada algoritmo debe reproducir la matriz esperada *exactamente*, y en el caso del ciclo negativo, ambos deben detectarlo. El Cuadro 1 resume las instancias; los archivos están en `tests/casos/`.

Instancia	n	m	Qué verifica	Origen de la solución
caso1_clrs	5	9	pesos negativos sin ciclo negativo; caminos óptimos no triviales	Fig. 25.1 de [1]
caso2_camino	4	3	pares inalcanzables (∞) en un camino dirigido	cálculo a mano
caso3_ciclo_negativo	3	3	detección de ciclo negativo (peso fraccionario 0.5)	cálculo a mano
caso4_fuertemente_conexo	4	6	ciclos de peso positivo; aristas directas peores que caminos indirectos	cálculo a mano

Cuadro 1: Instancias con solución conocida usadas como marco comparativo.

Batería 2: verificación cruzada aleatoria. El Algoritmo Base y Floyd-Warshall son implementaciones independientes basadas en técnicas distintas (relajación iterativa por aristas contra programación dinámica sobre intermedios); que ambas coincidan entrada a entrada sobre una colección grande y variada de instancias es evidencia fuerte de correctitud, pues un error en cualquiera de las dos difícilmente produciría exactamente las mismas matrices. Se generan 200 grafos aleatorios $G(n, p)$ reproducibles (generador `std::mt19937` con semilla fija 20260706), con n uniforme en $[1, 40]$, densidad $p \in \{0.1, 0.3, 0.6, 0.9\}$ rotando entre pruebas, y pesos múltiplos de 0.25 en $[0, 10]$ (pruebas pares) o en $[-3, 10]$ (pruebas impares, donde pueden aparecer ciclos negativos). Como los múltiplos de 0.25 son exactamente representables en `double`, las sumas de pesos son exactas y la comparación entre ambas matrices puede exigir igualdad exacta. El criterio de éxito por prueba es: ambos algoritmos coinciden en la detección de ciclo negativo y, en ausencia de este, sus matrices son idénticas (con ciclo negativo las distancias no están bien definidas y no se comparan).

2.3. Resultados de la verificación

La salida completa del programa es la siguiente:

```
== Bateria 1: instancias con solucion conocida (tests/casos) ==
[OK] caso1_clrs.grafo (n=5, m=9)
[OK] caso2_camino.grafo (n=4, m=3)
[OK] caso3_ciclo_negativo.grafo (n=3, m=3)
[OK] caso4_fuertemente_conexo.grafo (n=4, m=6)
== Bateria 2: verificacion cruzada sobre 200 grafos aleatorios ==
200/200 pruebas correctas (86 instancias con ciclo negativo,
detectado por ambos algoritmos)
== RESULTADO: todas las pruebas pasaron ==
```

Las cuatro instancias conocidas son reproducidas exactamente por ambos algoritmos, y en las 200 instancias aleatorias ambos coinciden por completo; en las 86 que contienen ciclos negativos, ambos los detectan. Las instrucciones para compilar y ejecutar la verificación están en el archivo `README.md` del repositorio; en síntesis:

```
g++ -std=c++17 -O2 -o verificar src/verificar.cpp
./verificar
```

3. Experimentos (pregunta 2.2)

3.1. Configuración experimental

Todos los experimentos se ejecutaron en la misma máquina: Windows 11, CPU AMD Ryzen 7 5800X (8 núcleos, 3.80 GHz), 24 GB RAM, compilador `g++` 16.1.0 (MinGW-w64, MSYS2), bandera `-O2`. Las llamadas a los algoritmos se protegen con `volatile` para evitar que el compilador las elimine del bucle de medición, incluso con optimizaciones habilitadas. Se eligió `-O2` en lugar del `-O0` sugerido por la suite `uhr` porque parte de la ventaja práctica de Floyd-Warshall reside justamente en las optimizaciones que habilita el compilador: su acceso secuencial a una matriz contigua permite excelente localidad de caché, loop unrolling y register allocation; medir con `-O0` habría eliminado artificialmente estas ventajas y distorsionado la comparación. Cada punto se midió con al menos 32 ejecuciones; los archivos CSV de salida contienen la media, la desviación estándar y los cinco cuartiles ($Q_0, \dots, Q_4$) en microsegundos. La semilla del generador aleatorio es fija (20260706) para garantizar reproducibilidad.

Se diseñaron cuatro experimentos que varían sistemáticamente la estructura del grafo, cubriendo los dos extremos del espectro de densidad y la presencia de pesos negativos. En todos los casos ambos algoritmos fueron ejecutados sobre exactamente el mismo grafo (verificación cruzada interna dentro de cada experimento).

3.2. Experimento 1: n creciente con densidad fija

Parámetros: $p = 0.3$ (densidad de aristas fija), pesos en $[0, 10]$, $n \in [10, 300]$ con step 10, 64 runs por punto.

Con $p = 0.3$, $m \approx 0.3n(n-1) = \Theta(n^2)$: el grafo es moderadamente denso. La teoría predice $t_{FW} = \Theta(n^3)$ y $t_{Base} = \Theta(n^2m) = \Theta(n^4)$. La Figura 1 muestra los resultados. El código fuente está en el Anexo A.4.

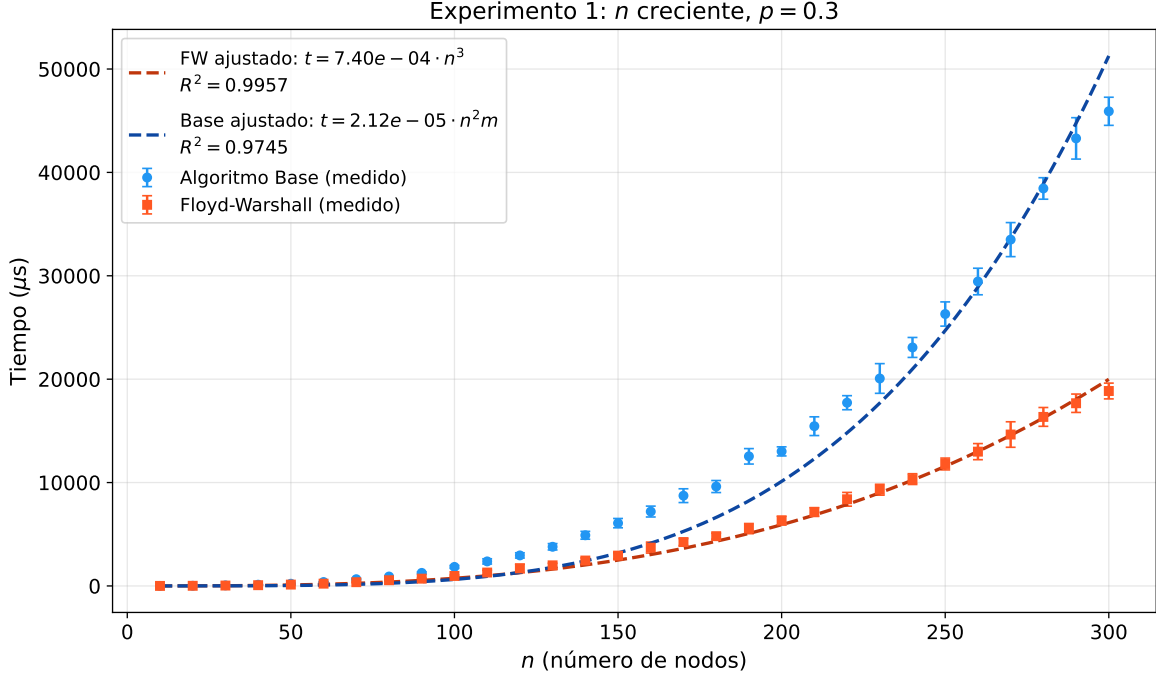


Figura 1: Experimento 1: n creciente, $p = 0.3$. Floyd-Warshall escala como n^3 ; el Algoritmo Base como n^4 . Las curvas punteadas son los ajustes por mínimos cuadrados.

3.3. Experimento 2: densidad creciente con n fijo

Parámetros: $n = 100$ fijo, $p \in [0.1, 1.0]$ con step 0.1, pesos en $[0, 10]$, 64 runs por punto.

Con $n = 100$, Floyd-Warshall ejecuta $\Theta(100^3)$ operaciones independientemente de m , por lo que su tiempo debe ser aproximadamente constante. El Algoritmo Base escala como $O(n^2m) = O(10^4 \cdot m)$, es decir lineal en m . La Figura 2 confirma ambas predicciones. Código fuente: Anexo A.5.

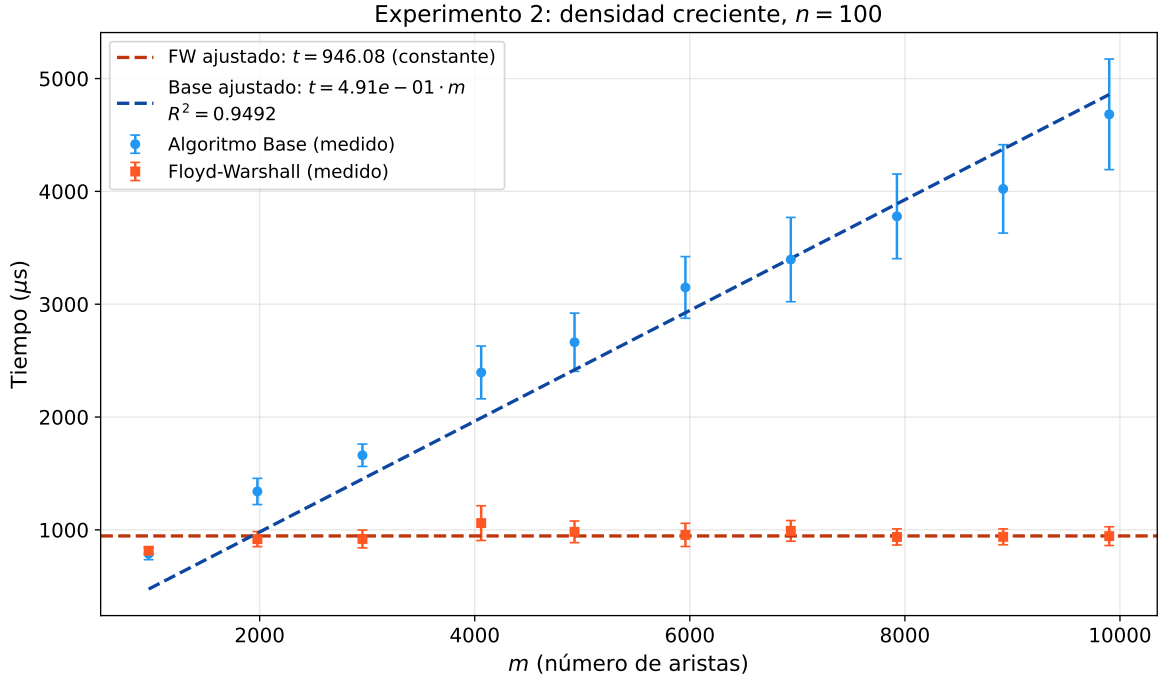


Figura 2: Experimento 2: densidad creciente, $n = 100$ fijo. Floyd-Warshall es constante; el Algoritmo Base crece linealmente con m .

3.4. Experimento 3: pesos negativos

Parámetros: $p = 0.15$, pesos en $[-2, 10]$, $n \in [10, 200]$ con step 20, 32 runs por punto.

Los pesos incluyen valores negativos para evaluar su impacto en el tiempo de Bellman-Ford. La densidad $p = 0.15$ se eligió como balance: con densidades mayores el Algoritmo Base se volvía demasiado lento incluso con -02 , mientras que con densidades mucho menores los grafos resultaban demasiado dispersos y los tiempos poco significativos. Con esta configuración la mayoría de los grafos contiene al menos un ciclo negativo para $n \geq 55$; no se filtran porque ambos algoritmos los detectan correctamente y porque descartar grafos con ciclos negativos habría requerido ejecutar Floyd-Warshall repetidas veces por cada n hasta encontrar uno libre de ciclos, volviendo el experimento impracticablemente lento para n grandes.

En cuanto a los resultados, con pesos no negativos (E1) Bellman-Ford suele alcanzar el punto fijo en pocas rondas y el early exit corta la ejecución; con pesos negativos, una arista negativa puede seguir mejorando distancias durante muchas más rondas (a veces las $n - 1$ completas), lo que anula el early exit y dispara el tiempo. Floyd-Warshall, en cambio, es insensible al signo de los pesos y ejecuta exactamente n^3 iteraciones en todos los casos. La Figura 3 muestra que, a diferencia del Experimento 1, la brecha entre ambos algoritmos se amplía significativamente por este efecto. Código fuente: Anexo A.6.

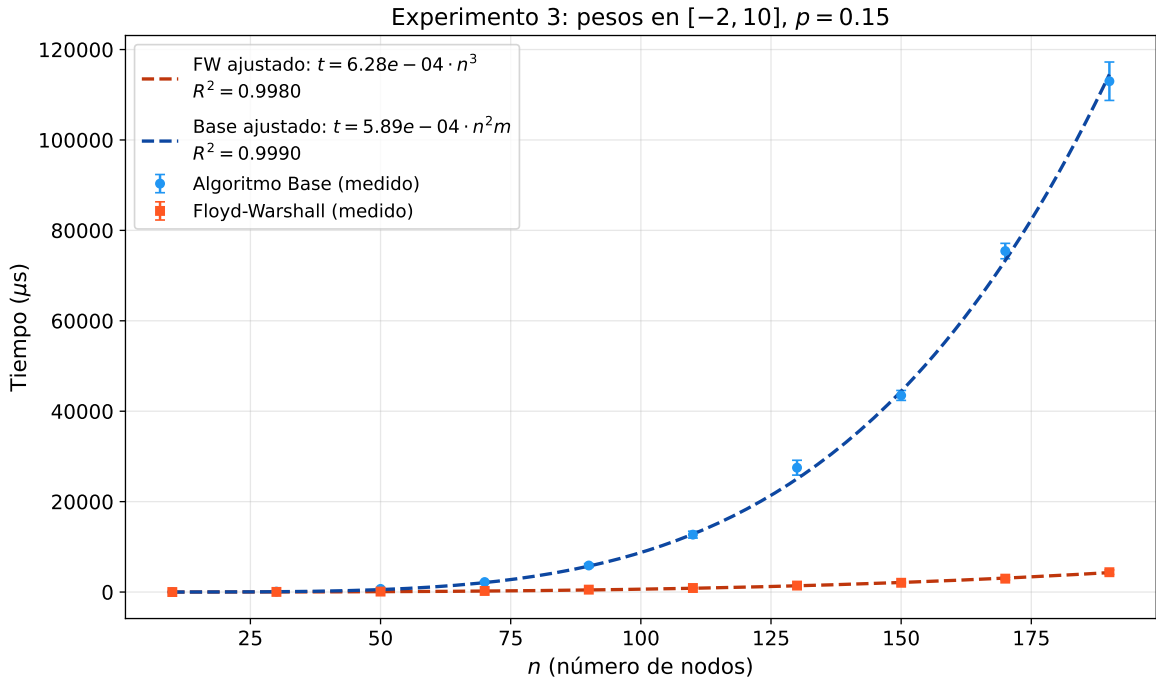


Figura 3: Experimento 3: pesos en $[-2, 10]$, $p = 0.15$. Los pesos negativos anulan el early exit de Bellman-Ford y amplían la brecha con Floyd-Warshall.

3.5. Experimento 4: grafos completos

Parámetros: $p = 1.0$ (grafo completo, $m = n(n-1)$), pesos en $[0, 10]$, $n \in [10, 200]$ con step 20, 32 runs por punto.

Este es el peor caso asintótico para el Algoritmo Base: con $m = n(n-1) = \Theta(n^2)$, su complejidad se vuelve $O(n^4)$ pura, mientras que Floyd-Warshall se mantiene en $\Theta(n^3)$. La brecha entre ambos es máxima, como ilustra la Figura 4. Código fuente: Anexo A.7.

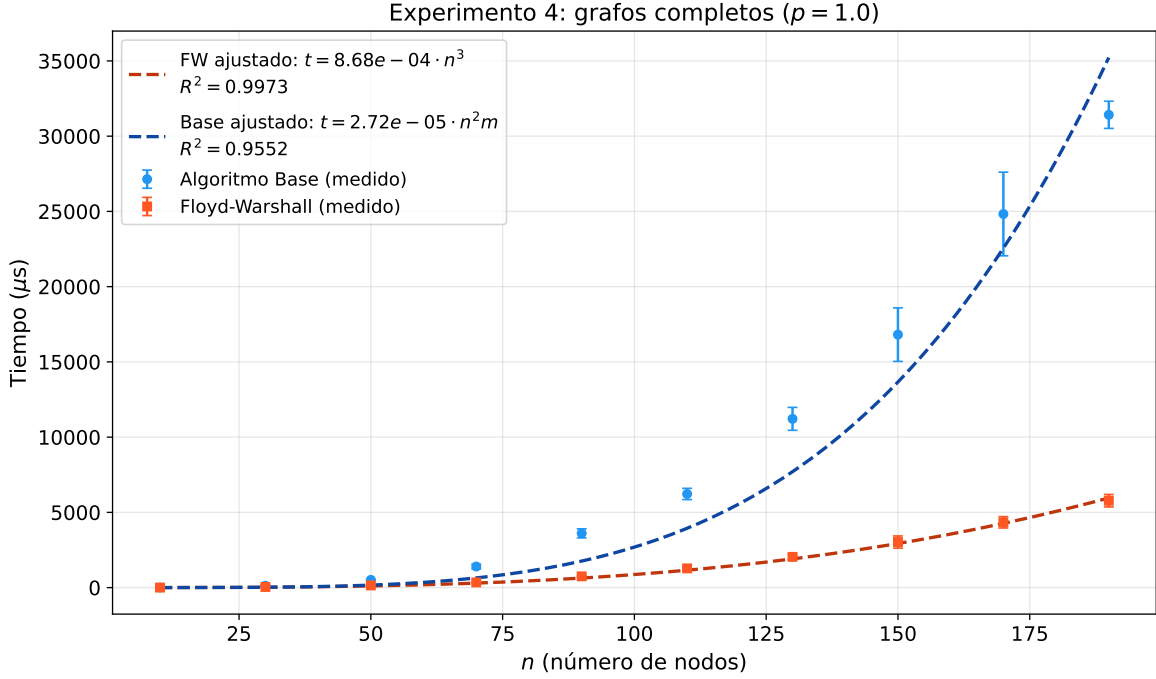


Figura 4: Experimento 4: grafos completos ($p = 1.0$). La brecha $O(n^4)$ vs. $\Theta(n^3)$ es máxima.

3.6. Generación de los gráficos

Los gráficos fueron generados con Python 3 (matplotlib) mediante el script `experimentos/graficar.py`. Cada gráfico incluye los puntos medidos con barras de error (± 1 desviación estándar) y las curvas de ajuste teórico calculadas por mínimos cuadrados con intercepto nulo; el coeficiente de determinación R^2 se reporta en la leyenda.

4. Datasets reales (pregunta 2.3)

Se ejecutó Floyd-Warshall sobre dos datasets reales obtenidos de Network Repository [2]. La Tabla 2 resume los resultados. Las matrices de distancia completas se escribieron en archivos CSV con formato `nodo1,nodo2,distancia`, donde la distancia es `INF` si no existe camino entre el par.

Dataset	n	m	Tiempo (s)	Ciclo negativo
bio-SC-TS	636	3 959	0.125	No
power-685-bus	685	3 249	0.157	Sí

Cuadro 2: Resultados de Floyd-Warshall sobre datasets reales.

bio-SC-TS es una red biológica de 636 nodos y 3 959 aristas (dirigida, sin pesos negativos). Floyd-Warshall procesa los $636^3 \approx 2.57 \times 10^8$ pasos del bucle interno en 0.125 s, consistente con la complejidad $\Theta(n^3)$ y potenciado por la compilación con `-O2`, que explota la localidad de caché del acceso secuencial a la matriz y optimiza el bucle interno mediante register allocation y loop unrolling.

power-685-bus es una matriz de admitancia de un sistema eléctrico de 685 barras, exportada en formato Matrix Market. Contiene pesos negativos que producen ciclos negativos; Floyd-Warshall los detecta correctamente ($D[i][i] < 0$ en la diagonal). Las entradas del CSV son mayoritariamente `INF` porque la presencia de un ciclo negativo invalida las distancias entre pares afectados. Código fuente: Anexo A.8.

5. Discusión y Extensiones

5.1. Discusión de los resultados experimentales (pregunta 3.1)

Los experimentos muestran una correspondencia clara entre la teoría y la ejecución práctica. En todos los casos las curvas de ajuste obtienen coeficientes R^2 cercanos a 1, lo que confirma que las cotas asintóticas $O(n^2m)$ para el Algoritmo Base y $\Theta(n^3)$ para Floyd-Warshall predicen correctamente el comportamiento observado.

Sin embargo, se aprecian varios fenómenos que la notación asintótica no captura:

1. **Constante multiplicativa y localidad de caché.** En el Experimento 1, Floyd-Warshall es consistentemente más rápido que el Algoritmo Base incluso para los n más pequeños, donde la diferencia asintótica aún no domina. Esto se debe a que el bucle interno de Floyd-Warshall solo realiza una suma, una comparación y una asignación sobre una matriz contigua en memoria (excelente localidad de caché), mientras que el Algoritmo Base recorre la lista de aristas n veces, con patrones de acceso a memoria menos regulares.
2. **Pesos negativos y early exit.** En el Experimento 3, la brecha entre ambos algoritmos es mucho mayor que en el Experimento 1. La causa es que Bellman-Ford implementa un early exit cuando una ronda completa no produce ninguna relajación. Con pesos no negativos, esto ocurre tras pocas rondas; con pesos negativos, el algoritmo suele necesitar las $n - 1$ rondas completas, perdiendo esa optimización. Floyd-Warshall no tiene este mecanismo y su tiempo es independiente del signo de los pesos.
3. **Efectos de caché.** En el Experimento 2, el tiempo de Floyd-Warshall no es perfectamente constante: muestra una ligera tendencia creciente con m . Esto no contradice $\Theta(n^3)$ (que es

independiente de m asintóticamente), pero refleja que la construcción de $D^{(0)}$ depende de m y que un mayor número de aristas puede afectar la localidad de los accesos durante las primeras iteraciones.

4. **Impacto de la optimización del compilador.** Para verificar que la elección de -02 era la correcta, se repitieron todos los experimentos con -00 (la bandera sugerida por uhr). La diferencia es contundente: Floyd-Warshall es entre 14 y 33 veces más rápido con -02, mientras que el Algoritmo Base apenas cambia (Tabla 3). Esto se debe a que el triple bucle de Floyd-Warshall sobre una matriz contigua es el caso ideal para loop unrolling, register allocation y vectorización; el Algoritmo Base, que recorre una lista de aristas con accesos impredecibles a memoria, no se beneficia de estas optimizaciones. Medir con -00 habría distorsionado la comparación al eliminar artificialmente una de las ventajas prácticas más importantes de Floyd-Warshall.

Experimento	FW con -00	FW con -02	Factor
E1 (coef. n^3)	1.35×10^{-2}	7.40×10^{-4}	18×
E2 (constante, μs)	13 260	946	14×
E3 (coef. n^3)	2.11×10^{-2}	6.28×10^{-4}	33×
E4 (coef. n^3)	1.46×10^{-2}	8.68×10^{-4}	17×

Cuadro 3: Comparación de Floyd-Warshall con -00 vs. -02. En E2 se compara la constante del ajuste; en E3 y E4 el coeficiente de n^3 .

5.2. Caso especial: pesos unitarios y otros contextos (pregunta 3.2)

Pesos unitarios. Si todos los pesos de las aristas son 1, el problema APSP se reduce a calcular la distancia (en número de aristas) entre todo par de nodos. En este caso existe una solución más rápida que Floyd-Warshall: ejecutar BFS desde cada nodo. Como BFS sobre un grafo no pesado cuesta $O(n+m)$, las n ejecuciones suman $O(n(n+m)) = O(n^2 + nm)$. En grafos dispersos ($m = O(n)$) esto es $O(n^2)$, un factor n mejor que Floyd-Warshall; en grafos densos ($m = \Theta(n^2)$) es $O(n^3)$, igual asintóticamente pero con mejores constantes (BFS usa colas, no aritmética de punto flotante).

Otros contextos donde se puede superar a Floyd-Warshall.

- **Grafos dispersos** ($m = O(n)$): el Algoritmo Base ya es $O(n^3)$ como Floyd-Warshall, pero ejecutar Dijkstra desde cada nodo (con pesos no negativos) da $O(n(m + n \log n))$, que en grafos dispersos es $O(n^2 \log n)$, mejor que $O(n^3)$.
- **Grafos sin ciclos** (DAGs): APSP se resuelve en $O(n(n+m)) = O(n^2 + nm)$ mediante orden topológico y programación dinámica.
- **Matrices de adyacencia densas con pesos pequeños**: el algoritmo de Williams [3] resuelve APSP en $\tilde{O}(n^3 / \log^2 n)$ usando el Method of Four Russians sobre el semianillo (mín, +).

5.3. Complejidad asintótica conocida y uso práctico (pregunta 3.3)

Mejor complejidad asintótica conocida. Para el problema APSP general (con pesos reales arbitrarios), el mejor algoritmo conocido a la fecha es el de Williams [3], que alcanza tiempo $\tilde{O}(n^3 / \log^2 n)$ mediante una combinación de las técnicas de Valiant (multiplicación de matrices sobre

el semianillo $(\min, +)$ y el método de los four russians. No se conoce un algoritmo subcúbico verdaderamente general (i.e., $O(n^{3-\varepsilon})$ para $\varepsilon > 0$) para APSP con pesos reales; de hecho, la hipótesis del APSP conjetura que no existe.

Para grafos con pesos enteros pequeños (en valor absoluto acotado por M), existen algoritmos verdaderamente subcúbicos: Zwick [4] da $\tilde{O}(Mn^\omega)$, donde $\omega < 2.372$ es el exponente de la multiplicación de matrices, aprovechando la reducción de APSP a multiplicación de matrices sobre $(\min, +)$.

Implementación en la práctica. En librerías reales de grafos, la versión que se utiliza no es la ingenua de triple bucle, sino una variante *bloqueada* (blocked Floyd-Warshall) que divide la matriz en bloques que caben en la caché L2/L3 del procesador, explotando al máximo la localidad espacial y temporal. Librerías como Boost Graph Library (C++) delegan APSP a n ejecuciones de Dijkstra cuando los pesos son no negativos y el grafo es disperso, reservando Floyd-Warshall bloqueado para grafos densos o cuando se requiere el camino más corto entre *todos* los pares sin excepción.

A. Códigos fuente

A.1. src/grafos.hpp

```
1 #pragma once
2 // grafos.hpp -- Representacion de grafos dirigidos con pesos reales.
3 // No se emplean librerias de grafos: solo contenedores estandar de C++.
4
5 #include <fstream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10 // Arista dirigida u -> v con peso w.
11 struct Arista {
12     int u;
13     int v;
14     double w;
15 };
16
17 // Grafo dirigido representado por su numero de nodos y su lista de aristas.
18 // Los nodos se identifican con los enteros 0, 1, ..., n-1.
19 struct Grafo {
20     int n = 0;
21     std::vector<Arista> aristas;
22 };
23
24 // Lee un grafo desde un archivo de texto con el formato:
25 //   n m
26 //   u v w      (m lineas: arista u -> v de peso w; nodos 0-indexados)
27 inline Grafo leer_grafo(const std::string& ruta) {
28     std::ifstream entrada(ruta);
29     if (!entrada) throw std::runtime_error("No se pudo abrir el archivo: " + ruta);
30     Grafo g;
31     int m = 0;
32     entrada >> g.n >> m;
33     g.aristas.reserve(m);
34     for (int i = 0; i < m; ++i) {
35         Arista a;
36         entrada >> a.u >> a.v >> a.w;
37         g.aristas.push_back(a);
38     }
39     if (!entrada) throw std::runtime_error("Formato invalido en: " + ruta);
40     return g;
41 }
```

A.2. src/apsp.hpp

```
1 #pragma once
2 // apsp.hpp -- Algoritmos para el problema de Distancia Minima entre todo par
3 // de nodos (APSP):
4 //   * Algoritmo Base: n ejecuciones de Bellman-Ford (pregunta 1.1).
5 //   * Algoritmo de Floyd-Warshall: programacion dinamica (pregunta 1.2).
6 // El diseno, correctitud y analisis de complejidad estan en el informe
7 // (secciones 1.1 y 1.2); la documentacion de la implementacion, en la
8 // seccion 2 del informe.
9
```

```

10 #include <limits>
11 #include <vector>
12
13 #include "grafo.hpp"
14
15 // Distancia "infinita": representa que no existe camino. La aritmetica IEEE
16 // de double garantiza  $INF + c == INF$  (para  $c$  finito) e  $INF + INF == INF$ , y
17 // toda comparacion " $INF + c < x$ " con  $x$  finito es falsa; por lo tanto las
18 // relajaciones son seguras sin tratar casos especiales.
19 const double INF = std::numeric_limits<double>::infinity();
20
21 // Matriz de distancias:  $D[i][j]$  = distancia minima desde  $i$  hasta  $j$ .
22 using Matriz = std::vector<std::vector<double>>>;
23
24 // -----
25 // Bellman-Ford (una fuente).
26 // Calcula  $dist[v]$  = distancia minima desde  $s$  hasta  $v$  para todo  $v$ , en  $O(n*m)$ :
27 // a lo mas  $n-1$  rondas en las que se intenta relajar cada arista. Si en una
28 // ronda ninguna distancia mejora, el algoritmo ya alcanzo un punto fijo y
29 // termina antes (esto no afecta la correctitud, solo el tiempo). Una ronda
30 // adicional detecta ciclos negativos alcanzables desde  $s$ : si alguna arista
31 // aun puede relajarse tras  $n-1$  rondas, existe uno.
32 // Si  $ciclo\_negativo \neq nullptr$ , se escribe alli el resultado de la deteccion.
33 // -----
34 inline std::vector<double> bellman_ford(const Grafo& g, int s,
35                                         bool* ciclo_negativo = nullptr) {
36     std::vector<double> dist(g.n, INF);
37     dist[s] = 0.0;
38     for (int ronda = 1; ronda <= g.n - 1; ++ronda) {
39         bool mejoro = false;
40         for (const Arista& a : g.aristas) {
41             if (dist[a.u] + a.w < dist[a.v]) {
42                 dist[a.v] = dist[a.u] + a.w;
43                 mejoro = true;
44             }
45         }
46         if (!mejoro) break; // punto fijo alcanzado: terminar antes
47     }
48     if (ciclo_negativo != nullptr) {
49         *ciclo_negativo = false;
50         for (const Arista& a : g.aristas) {
51             if (dist[a.u] + a.w < dist[a.v]) {
52                 *ciclo_negativo = true;
53                 break;
54             }
55         }
56     }
57     return dist;
58 }
59
60 // -----
61 // Algoritmo Base (pregunta 1.1).
62 // Resuelve APSP con exactamente  $n$  llamadas a Bellman-Ford, una por cada
63 // fuente: la fila  $s$  de la matriz resultado es la salida de  $bellman\_ford(g,s)$ .
64 // Complejidad:  $O(n^2 * m)$  en tiempo;  $\Theta(n^2)$  en espacio (la salida).
65 // Si  $ciclo\_negativo \neq nullptr$ , se escribe true si alguna llamada detecto un
66 // ciclo negativo (todo ciclo negativo es alcanzable desde sus propios nodos,
67 // de modo que al recorrer todas las fuentes ninguno pasa inadvertido).
68 // -----

```

```

69 inline Matriz algoritmo_base(const Grafo& g, bool* ciclo_negativo = nullptr) {
70     if (ciclo_negativo != nullptr) *ciclo_negativo = false;
71     Matriz D(g.n);
72     for (int s = 0; s < g.n; ++s) {
73         bool cn = false;
74         D[s] = bellman_ford(g, s, &cn);
75         if (cn && ciclo_negativo != nullptr) *ciclo_negativo = true;
76     }
77     return D;
78 }
79
80 // -----
81 // Algoritmo de Floyd-Warshall (pregunta 1.2).
82 // Programacion dinamica sobre el conjunto de vertices intermedios permitidos:
83 //  $D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$ ,
84 // con  $D_0$  la matriz de pesos directos. La actualizacion se realiza in situ
85 // sobre una unica matriz: en la iteracion k la fila k y la columna k no
86 // cambian (Lema demostrado en la seccion 1.2 del informe), por lo que leer
87 // entradas ya actualizadas equivale a leer las de la etapa k-1.
88 // Complejidad:  $\Theta(n^3)$  en tiempo y  $\Theta(n^2)$  en espacio.
89 // Si ciclo_negativo != nullptr, se escribe true si existe un ciclo negativo,
90 // lo que ocurre si y solo si alguna entrada diagonal termina negativa.
91 // -----
92 inline Matriz floyd_warshall(const Grafo& g, bool* ciclo_negativo = nullptr) {
93     // Caso base  $D_0$ : 0 en la diagonal, peso de la arista directa si existe
94     // (si hay aristas paralelas se conserva la de menor peso), INF si no.
95     Matriz D(g.n, std::vector<double>(g.n, INF));
96     for (int i = 0; i < g.n; ++i) D[i][i] = 0.0;
97     for (const Arista& a : g.aristas) {
98         if (a.w < D[a.u][a.v]) D[a.u][a.v] = a.w;
99     }
100
101     for (int k = 0; k < g.n; ++k) {
102         for (int i = 0; i < g.n; ++i) {
103             const double dik = D[i][k]; // invariante durante la iteracion k
104             for (int j = 0; j < g.n; ++j) {
105                 if (dik + D[k][j] < D[i][j]) D[i][j] = dik + D[k][j];
106             }
107         }
108     }
109
110     if (ciclo_negativo != nullptr) {
111         *ciclo_negativo = false;
112         for (int i = 0; i < g.n; ++i) {
113             if (D[i][i] < 0) {
114                 *ciclo_negativo = true;
115                 break;
116             }
117         }
118     }
119     return D;
120 }

```


A.3. src/verificar.cpp

```
1 // verificar.cpp -- Verificacion de correctitud de las implementaciones
2 // (pregunta 2.1). Ejecuta dos baterias de pruebas:
3 //
4 //   Bateria 1: instancias con solucion conocida. Cada archivo
5 //     tests/casos/X.grafo tiene un tests/casos/X.esperado con la matriz de
6 //     distancias correcta (calculada a mano o tomada de la literatura; ver
7 //     informe, seccion 2.3) o la palabra CICLO_NEGATIVO si la instancia
8 //     contiene un ciclo de peso negativo. Ambos algoritmos deben reproducir
9 //     exactamente lo esperado.
10 //
11 //   Bateria 2: verificacion cruzada sobre grafos aleatorios reproducibles
12 //     (semilla fija). El Algoritmo Base y Floyd-Warshall usan tecnicas
13 //     distintas; que coincidan entrada a entrada en cientos de instancias
14 //     variadas es evidencia fuerte de correctitud de ambos.
15 //
16 // Uso (desde la raiz del repositorio):
17 //   ./verificar [directorio_de_casos]      (por defecto "tests/casos")
18 //
19 //Codigo de salida: 0 si todas las pruebas pasan, 1 en caso contrario.
20
21 #include <algorithm>
22 #include <cmath>
23 #include <cstdio>
24 #include <filesystem>
25 #include <random>
26 #include <stdexcept>
27 #include <string>
28 #include <vector>
29
30 #include "apsp.hpp"
31
32 namespace fs = std::filesystem;
33
34 // Tolerancia al comparar distancias. Los pesos de todas las pruebas son
35 // multiplos de 0.25, exactamente representables en double, de modo que las
36 // sumas son exactas y en la practica la diferencia debe ser exactamente 0;
37 // la tolerancia queda solo como resguardo.
38 const double TOL = 1e-9;
39
40 bool iguales(double a, double b) {
41     if (std::isinf(a) || std::isinf(b)) return std::isinf(a) && std::isinf(b);
42     return std::fabs(a - b) <= TOL;
43 }
44
45 bool matrices_iguales(const Matriz& A, const Matriz& B) {
46     if (A.size() != B.size()) return false;
47     for (std::size_t i = 0; i < A.size(); ++i)
48         for (std::size_t j = 0; j < A.size(); ++j)
49             if (!iguales(A[i][j], B[i][j])) return false;
50     return true;
51 }
52
53 // Resultado esperado de una instancia: o bien "hay ciclo negativo", o bien
54 // una matriz de n x n donde el token INF denota "no existe camino".
55 struct Esperado {
56     bool ciclo_negativo = false;
57     Matriz D;
```

```

58 };
59
60 Esperado leer_esperado(const std::string& ruta, int n) {
61     std::ifstream entrada(ruta);
62     if (!entrada) throw std::runtime_error("No se pudo abrir el archivo: " + ruta);
63     Esperado e;
64     std::string tok;
65     entrada >> tok;
66     if (tok == "CICLO_NEGATIVO") {
67         e.ciclo_negativo = true;
68         return e;
69     }
70     e.D.assign(n, std::vector<double>(n));
71     for (int i = 0; i < n; ++i) {
72         for (int j = 0; j < n; ++j) {
73             if (i != 0 || j != 0) entrada >> tok; // el primer token ya fue leído
74             e.D[i][j] = (tok == "INF") ? INF : std::stod(tok);
75         }
76     }
77     if (!entrada) throw std::runtime_error("Formato invalido en: " + ruta);
78     return e;
79 }
80
81 int main(int argc, char** argv) {
82     const std::string dir = (argc > 1) ? argv[1] : "tests/casos";
83     int fallas = 0;
84
85     // ----- Bateria 1: instancias con solucion conocida -----
86     std::printf("== Bateria 1: instancias con solucion conocida (%s) ==\n", dir.
87         c_str());
88     std::vector<fs::path> casos;
89     for (const auto& entrada : fs::directory_iterator(dir))
90         if (entrada.path().extension() == ".grafo") casos.push_back(entrada.path());
91
92     std::sort(casos.begin(), casos.end());
93
94     for (const fs::path& ruta : casos) {
95         Grafo g = leer_grafo(ruta.string());
96         fs::path ruta_esperado = ruta;
97         ruta_esperado.replace_extension(".esperado");
98         Esperado esperado = leer_esperado(ruta_esperado.string(), g.n);
99
100         bool cn_base = false, cn_fw = false;
101         Matriz D_base = algoritmo_base(g, &cn_base);
102         Matriz D_fw = floyd_warshall(g, &cn_fw);
103
104         bool ok;
105         if (esperado.ciclo_negativo) {
106             // Ambos algoritmos deben detectar el ciclo negativo.
107             ok = cn_base && cn_fw;
108         } else {
109             // Ninguno debe reportar ciclo negativo y ambas matrices deben
110             // coincidir con la esperada (y, en consecuencia, entre si).
111             ok = !cn_base && !cn_fw && matrices_iguales(D_base, esperado.D) &&
112                 matrices_iguales(D_fw, esperado.D);
113         }
114         std::printf(" [%s] %s (n=%d, m=%d)\n", ok ? "OK" : "FALLA",
115             ruta.filename().string().c_str(), g.n,
116             static_cast<int>(g.aristas.size()));

```

```

115         if (!ok) ++fallas;
116     }
117
118     // ----- Bateria 2: verificacion cruzada aleatoria -----
119     const int PRUEBAS = 200;
120     std::printf("== Bateria 2: verificacion cruzada sobre %d grafos aleatorios ==\n",
121         PRUEBAS);
122     std::mt19937 rng(20260706u); // semilla fija: pruebas reproducibles
123     const double DENSIDADES[] = {0.1, 0.3, 0.6, 0.9};
124     int con_ciclo = 0, fallas_aleatorias = 0;
125
126     for (int t = 0; t < PRUEBAS; ++t) {
127         // Grafo aleatorio G(n, p): cada arista (u,v), u != v, existe con
128         // probabilidad p. Pruebas pares: pesos en [0, 10]; pruebas impares:
129         // pesos en [-3, 10] (pueden aparecer ciclos negativos, que ambos
130         // algoritmos deben senalar). Los pesos son multiples de 0.25.
131         int n = std::uniform_int_distribution<int>(1, 40)(rng);
132         double p = DENSIDADES[t % 4];
133         int peso_minimo = (t % 2 == 0) ? 0 : -12; // en cuartos
134         std::bernoulli_distribution existe_arista(p);
135         std::uniform_int_distribution<int> cuartos(peso_minimo, 40);
136
137         Grafo g;
138         g.n = n;
139         for (int u = 0; u < n; ++u)
140             for (int v = 0; v < n; ++v)
141                 if (u != v && existe_arista(rng))
142                     g.aristas.push_back({u, v, cuartos(rng) / 4.0});
143
144         bool cn_base = false, cn_fw = false;
145         Matriz D_base = algoritmo_base(g, &cn_base);
146         Matriz D_fw = floyd_warshall(g, &cn_fw);
147
148         // Ambos deben coincidir en la deteccion de ciclos negativos; si no
149         // hay ciclo negativo, las matrices deben ser identicas (con ciclo
150         // negativo las distancias no estan bien definidas y no se comparan).
151         bool ok = (cn_base == cn_fw) && (cn_base || matrices_iguales(D_base, D_fw));
152
153         if (cn_base && cn_fw) ++con_ciclo;
154         if (!ok) {
155             std::printf(" [FALLA] prueba aleatoria %d: n=%d, p=%.1f, m=%d\n", t, n,
156                 p, static_cast<int>(g.aristas.size()));
157             ++fallas_aleatorias;
158         }
159     }
160     std::printf(" %d/%d pruebas correctas (%d instancias con ciclo negativo, "
161         "detectado por ambos algoritmos)\n",
162         PRUEBAS - fallas_aleatorias, PRUEBAS, con_ciclo);
163     fallas += fallas_aleatorias;
164
165     if (fallas == 0)
166         std::printf("== RESULTADO: todas las pruebas pasaron ==\n");
167     else
168         std::printf("== RESULTADO: %d prueba(s) FALLARON ==\n", fallas);
169     return (fallas == 0) ? 0 : 1;

```

A.4. experimentos/exp1_n_creciente.cpp

```
1 // exp1_n_creciente.cpp
2 // Experimento 1 (Pregunta 2.2)
3 // n creciente, densidad fija p = 0.3, pesos en [0, 10].
4 //
5 // Uso: ./exp1 <runs> <lower> <upper> <step>
6 // Ej: ./exp1 64 10 300 10
7 // Salida: resultados/base_e1_us.csv, resultados/fw_e1_us.csv
8
9 #include <chrono>
10 #include <cmath>
11 #include <fstream>
12 #include <iostream>
13 #include <random>
14 #include <string>
15 #include <vector>
16
17 #include "../src/apsp.hpp"
18 #include "../src/grafico.hpp"
19 #include "experimento_utils.hpp"
20
21 int main(int argc, char* argv[]) {
22     std::int64_t runs, lower, upper, step;
23     validate_input(argc, argv, runs, lower, upper, step);
24
25     // --- Parametros del experimento ---
26     const double DENSIDAD = 0.3; // p = 0.3: balance visibilidad/rendimiento
27     const double PESO_MIN = 0.0;
28     const double PESO_MAX = 10.0;
29     const unsigned SEMILLA = 20260706u; // fija para reproducibilidad
30
31     std::mt19937_64 rng(SEMILLA);
32     std::uniform_real_distribution<double> peso_dist(PESO_MIN, PESO_MAX);
33
34     // Archivos de salida
35     std::ofstream csv_base("resultados/base_e1_us.csv");
36     std::ofstream csv_fw("resultados/fw_e1_us.csv");
37     auto escribir_encabezado = [](std::ofstream& f) {
38         f << "n,t_mean,t_stdev,t_Q0,t_Q1,t_Q2,t_Q3,t_Q4\n";
39     };
40     escribir_encabezado(csv_base);
41     escribir_encabezado(csv_fw);
42
43     const std::int64_t total_pruebas = ((upper - lower) / step + 1);
44     const std::int64_t total_runs = runs * total_pruebas;
45
46     std::vector<double> times(runs);
47     std::vector<double> q;
48     double mean_time, time_stdev, dev;
49
50     using Clock = std::chrono::high_resolution_clock;
51     using Dur = std::chrono::duration<double, std::micro>; // microsegundos
52
53     std::cerr << "\033[0;36mExperimento 1: n creciente, p=" << DENSIDAD
54         << " (n=" << lower << ".." << upper << ", step=" << step
55         << ", runs=" << runs << ")\033[0m\n\n";
56
57     std::int64_t ejecutadas = 0;
```

```

58 for (std::int64_t n = lower; n <= upper; n += step) {
59     // --- Generar grafo G(n, p) ---
60     std::bernoulli_distribution arista_dist(DENSIDAD);
61     Grafo g;
62     g.n = static_cast<int>(n);
63     for (int u = 0; u < g.n; ++u)
64         for (int v = 0; v < g.n; ++v)
65             if (u != v && arista_dist(rng))
66                 g.aristas.push_back({u, v, peso_dist(rng)});
67
68     // --- Verificacion cruzada: ambos algoritmos deben coincidir ---
69     {
70         bool cn_base = false, cn_fw = false;
71         Matriz D_base = algoritmo_base(g, &cn_base);
72         Matriz D_fw = floyd_warshall(g, &cn_fw);
73         if (cn_base != cn_fw) {
74             std::cerr << "\n [ERROR] n=" << n
75                 << ": los algoritmos discrepan\n";
76             std::exit(EXIT_FAILURE);
77         }
78     }
79
80     // ----- Medir algoritmo_base (n ejecuciones de Bellman-Ford) -----
81     mean_time = 0.0;
82     time_stdev = 0.0;
83     for (std::int64_t i = 0; i < runs; ++i) {
84         display_progress(++ejecutadas, total_runs * 2);
85
86         auto t0 = Clock::now();
87         // volatile evita que el compilador elimine la llamada
88         volatile auto D = algoritmo_base(g);
89         (void)D;
90         auto t1 = Clock::now();
91
92         times[i] = Dur(t1 - t0).count();
93         mean_time += times[i];
94     }
95     mean_time /= runs;
96     for (std::int64_t i = 0; i < runs; ++i) {
97         dev = times[i] - mean_time;
98         time_stdev += dev * dev;
99     }
100     time_stdev = std::sqrt(time_stdev / (runs - 1));
101     quartiles(times, q);
102     csv_base << n << ", " << mean_time << ", " << time_stdev << ", "
103         << q[0] << ", " << q[1] << ", " << q[2] << ", " << q[3] << ", "
104         << q[4] << "\n";
105
106     // ----- Medir floyd_warshall (programacion dinamica, Theta(n^3)) -----
107     mean_time = 0.0;
108     time_stdev = 0.0;
109     for (std::int64_t i = 0; i < runs; ++i) {
110         display_progress(++ejecutadas, total_runs * 2);
111
112         auto t0 = Clock::now();
113         volatile auto D = floyd_warshall(g);
114         (void)D;
115         auto t1 = Clock::now();
116

```

```

117         times[i] = Dur(t1 - t0).count();
118         mean_time += times[i];
119     }
120     mean_time /= runs;
121     for (std::int64_t i = 0; i < runs; ++i) {
122         dev = times[i] - mean_time;
123         time_stdev += dev * dev;
124     }
125     time_stdev = std::sqrt(time_stdev / (runs - 1));
126     quartiles(times, q);
127     csv_fw << n << "," << mean_time << "," << time_stdev << ","
128         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
129         << q[4] << "\n";
130 }
131
132 std::cerr << "\n\n033[1;32mExperimento 1 completado.\033[0m\n";
133 csv_base.close();
134 csv_fw.close();
135 return 0;
136 }

```

A.5. experimentos/exp2_densidad.cpp

```

1 // exp2_densidad.cpp
2 // Experimento 2 (Pregunta 2.2)
3 // Densidad creciente, n = 100 fijo, pesos en [0, 10].
4 //
5 // Uso: ./exp2 <runs> <lower> <upper> <step>
6 //       lower/upper/step = densidad * 100 (ej: 30 = p 0.30)
7 // Ej: ./exp2 64 10 100 10
8 // Salida: resultados/base_e2-us.csv, resultados/fw_e2-us.csv
9
10 #include <chrono>
11 #include <cmath>
12 #include <fstream>
13 #include <iostream>
14 #include <random>
15 #include <string>
16 #include <vector>
17
18 #include "../src/apsp.hpp"
19 #include "../src/grafico.hpp"
20 #include "experimento_utils.hpp"
21
22 int main(int argc, char* argv[]) {
23     std::int64_t runs, lower, upper, step;
24     validate_input(argc, argv, runs, lower, upper, step);
25
26     // --- Parametros del experimento ---
27     const int N_FIJO = 100; // n fijo: FW es constante, Base crece con m
28     const double PESO_MIN = 0.0;
29     const double PESO_MAX = 10.0;
30     const unsigned SEMILLA = 20260706u; // fija para reproducibilidad
31
32     std::mt19937_64 rng(SEMILLA);
33     std::uniform_real_distribution<double> peso_dist(PESO_MIN, PESO_MAX);
34 }

```

```

35 // Archivos de salida
36 std::ofstream csv_base("resultados/base_e2_us.csv");
37 std::ofstream csv_fw("resultados/fw_e2_us.csv");
38 auto escribir_encabezado = [](std::ofstream& f) {
39     f << "densidad,m,t_mean,t_stdev,t_Q0,t_Q1,t_Q2,t_Q3,t_Q4\n";
40 };
41 escribir_encabezado(csv_base);
42 escribir_encabezado(csv_fw);
43
44 const std::int64_t total_pruebas = ((upper - lower) / step + 1);
45 const std::int64_t total_runs = runs * total_pruebas;
46
47 std::vector<double> times(runs);
48 std::vector<double> q;
49 double mean_time, time_stdev, dev;
50 using Clock = std::chrono::high_resolution_clock;
51 using Dur = std::chrono::duration<double, std::micro>;
52
53 std::cerr << "\033[0;36mExperimento 2: densidad creciente, n=" << N_FIJO
54 << " (p=" << lower/100.0 << ".." << upper/100.0
55 << ", step=" << step/100.0 << ", runs=" << runs << ")\033[0m\n\n";
56
57 std::int64_t ejecutadas = 0;
58 for (std::int64_t pct = lower; pct <= upper; pct += step) {
59     double densidad = pct / 100.0;
60
61     // --- Generar grafo G(n, p) con n fijo ---
62     std::bernoulli_distribution arista_dist(densidad);
63     Grafo g;
64     g.n = N_FIJO;
65     for (int u = 0; u < g.n; ++u)
66         for (int v = 0; v < g.n; ++v)
67             if (u != v && arista_dist(rng))
68                 g.aristas.push_back({u, v, peso_dist(rng)});
69
70     // --- Verificacion cruzada: ambos algoritmos deben coincidir ---
71     {
72         bool cn_base = false, cn_fw = false;
73         Matriz D_base = algoritmo_base(g, &cn_base);
74         Matriz D_fw = floyd_warshall(g, &cn_fw);
75         if (cn_base != cn_fw) {
76             std::cerr << "\n [ERROR] p=" << densidad
77 << ": los algoritmos discrepan\n";
78             std::exit(EXIT_FAILURE);
79         }
80     }
81
82     // ==== Medir algoritmo_base (n ejecuciones de Bellman-Ford) ====
83     mean_time = 0.0;
84     time_stdev = 0.0;
85     for (std::int64_t i = 0; i < runs; ++i) {
86         display_progress(++ejecutadas, total_runs * 2);
87         auto t0 = Clock::now();
88         // volatile evita que el compilador elimine la llamada
89         volatile auto D = algoritmo_base(g);
90         (void)D;
91         auto t1 = Clock::now();
92         times[i] = Dur(t1 - t0).count();
93         mean_time += times[i];

```

```

94     }
95     mean_time /= runs;
96     for (std::int64_t i = 0; i < runs; ++i) {
97         dev = times[i] - mean_time;
98         time_stdev += dev * dev;
99     }
100     time_stdev = std::sqrt(time_stdev / (runs - 1));
101     quartiles(times, q);
102     csv_base << densidad << "," << g.aristas.size() << ","
103         << mean_time << "," << time_stdev << ","
104         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
105         << q[4] << "\n";
106
107     // ===== Medir floyd_warshall (programacion dinamica, Theta(n^3)) =====
108     mean_time = 0.0;
109     time_stdev = 0.0;
110     for (std::int64_t i = 0; i < runs; ++i) {
111         display_progress(++ejecutadas, total_runs * 2);
112         auto t0 = Clock::now();
113         volatile auto D = floyd_warshall(g);
114         (void)D;
115         auto t1 = Clock::now();
116         times[i] = Dur(t1 - t0).count();
117         mean_time += times[i];
118     }
119     mean_time /= runs;
120     for (std::int64_t i = 0; i < runs; ++i) {
121         dev = times[i] - mean_time;
122         time_stdev += dev * dev;
123     }
124     time_stdev = std::sqrt(time_stdev / (runs - 1));
125     quartiles(times, q);
126     csv_fw << densidad << "," << g.aristas.size() << ","
127         << mean_time << "," << time_stdev << ","
128         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
129         << q[4] << "\n";
130 }
131
132 std::cerr << "\n\n033[1;32mExperimento 2 completado.\033[0m\n";
133 csv_base.close();
134 csv_fw.close();
135 return 0;
136 }

```


A.6. experimentos/exp3_pesos_negativos.cpp

```
1 // exp3_pesos_negativos.cpp
2 // Experimento 3 (Pregunta 2.2)
3 // n creciente, densidad fija  $p = 0.15$ , pesos en  $[-2, 10]$ .
4 //
5 // Uso: ./exp3 <runs> <lower> <upper> <step>
6 // Ej: ./exp3 32 10 200 20
7 // Salida: resultados/base_e3_us.csv, resultados/fw_e3_us.csv
8
9 #include <chrono>
10 #include <cmath>
11 #include <fstream>
12 #include <iostream>
13 #include <random>
14 #include <string>
15 #include <vector>
16
17 #include "../src/apsp.hpp"
18 #include "../src/grafico.hpp"
19 #include "experimento_utils.hpp"
20
21 int main(int argc, char *argv[])
22 {
23     std::int64_t runs, lower, upper, step;
24     validate_input(argc, argv, runs, lower, upper, step);
25
26     // --- Parametros del experimento ---
27     const double DENSIDAD = 0.15; //  $p = 0.15$ : balance dispersion/rendimiento
28     const double PESO_MIN = -2.0; // negativos para probar sensibilidad de BF
29     const double PESO_MAX = 10.0;
30     const unsigned SEMILLA = 20260706u; // fija para reproducibilidad
31
32     std::mt19937_64 rng(SEMILLA);
33     std::uniform_real_distribution<double> peso_dist(PESO_MIN, PESO_MAX);
34
35     // Archivos de salida
36     std::ofstream csv_base("resultados/base_e3_us.csv");
37     std::ofstream csv_fw("resultados/fw_e3_us.csv");
38     auto escribir_encabezado = [](std::ofstream &f)
39     {
40         f << "n,t_mean,t_stdev,t_Q0,t_Q1,t_Q2,t_Q3,t_Q4\n";
41     };
42     escribir_encabezado(csv_base);
43     escribir_encabezado(csv_fw);
44
45     const std::int64_t total_pruebas = ((upper - lower) / step + 1);
46     const std::int64_t total_runs = runs * total_pruebas;
47
48     std::vector<double> times(runs);
49     std::vector<double> q;
50     double mean_time, time_stdev, dev;
51     using Clock = std::chrono::high_resolution_clock;
52     using Dur = std::chrono::duration<double, std::micro>;
53
54     std::cerr << "\033[0;36mExperimento 3: pesos negativos, p=" << DENSIDAD
55               << ", pesos=[" << PESO_MIN << ", " << PESO_MAX
56               << "]" (n=" << lower << "..." << upper << ", step=" << step
57               << ", runs=" << runs << ")\033[0m\n\n";
```

```

58
59 std::int64_t ejecutadas = 0;
60 for (std::int64_t n = lower; n <= upper; n += step)
61 {
62     // --- Generar grafo aleatorio G(n, p) con pesos en [PESO_MIN, PESO_MAX]
63     ---
64     std::bernoulli_distribution arista_dist(DENSIDAD);
65     Grafo g;
66     g.n = static_cast<int>(n);
67     for (int u = 0; u < g.n; ++u)
68         for (int v = 0; v < g.n; ++v)
69             if (u != v && arista_dist(rng))
70                 g.aristas.push_back({u, v, peso_dist(rng)});
71
72     // --- Verificacion cruzada: ambos algoritmos deben coincidir ---
73     {
74         bool cn_base = false, cn_fw = false;
75         Matriz D_base = algoritmo_base(g, &cn_base);
76         Matriz D_fw = floyd_warshall(g, &cn_fw);
77         if (cn_base != cn_fw) {
78             std::cerr << "\n [ERROR] n=" << n
79                 << ": los algoritmos discrepan en deteccion de ciclo\n";
80             std::exit(EXIT_FAILURE);
81         }
82     }
83
84     // ----- Medir algoritmo_base (n ejecuciones de Bellman-Ford) -----
85     mean_time = 0.0;
86     time_stdev = 0.0;
87     for (std::int64_t i = 0; i < runs; ++i)
88     {
89         display_progress(++ejecutadas, total_runs * 2);
90         auto t0 = Clock::now();
91         // volatile evita que el compilador elimine la llamada
92         volatile auto D = algoritmo_base(g);
93         (void)D;
94         auto t1 = Clock::now();
95         times[i] = Dur(t1 - t0).count();
96         mean_time += times[i];
97     }
98     mean_time /= runs;
99     for (std::int64_t i = 0; i < runs; ++i)
100     {
101         dev = times[i] - mean_time;
102         time_stdev += dev * dev;
103     }
104     time_stdev = std::sqrt(time_stdev / (runs - 1));
105     quartiles(times, q);
106     csv_base << n << "," << mean_time << "," << time_stdev << ","
107         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
108         << q[4] << "\n";
109
110     // ----- Medir floyd_warshall (programacion dinamica, Theta(n^3)) -----
111     mean_time = 0.0;
112     time_stdev = 0.0;
113     for (std::int64_t i = 0; i < runs; ++i)
114     {
115         display_progress(++ejecutadas, total_runs * 2);
116         auto t0 = Clock::now();

```

```

116         volatile auto D = floyd_warshall(g);
117         (void)D;
118         auto t1 = Clock::now();
119         times[i] = Dur(t1 - t0).count();
120         mean_time += times[i];
121     }
122     mean_time /= runs;
123     for (std::int64_t i = 0; i < runs; ++i)
124     {
125         dev = times[i] - mean_time;
126         time_stdev += dev * dev;
127     }
128     time_stdev = std::sqrt(time_stdev / (runs - 1));
129     quartiles(times, q);
130     csv_fw << n << "," << mean_time << "," << time_stdev << ","
131             << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
132             << q[4] << "\n";
133 }
134
135 std::cerr << "\n\n033[1;32mExperimento 3 completado.\033[0m\n";
136 csv_base.close();
137 csv_fw.close();
138 return 0;
139 }

```

A.7. experimentos/exp4_grafos_completos.cpp

```

1 // exp4_grafos_completos.cpp
2 // Experimento 4 (Pregunta 2.2)
3 // Grafos completos (p = 1.0), pesos en [0, 10].
4 //
5 // Uso: ./exp4 <runs> <lower> <upper> <step>
6 // Ej: ./exp4 32 10 200 20
7 // Salida: resultados/base_e4_us.csv, resultados/fw_e4_us.csv
8
9 #include <chrono>
10 #include <cmath>
11 #include <fstream>
12 #include <iostream>
13 #include <random>
14 #include <string>
15 #include <vector>
16
17 #include "../src/apsp.hpp"
18 #include "../src/grafos.hpp"
19 #include "experimento_utils.hpp"
20
21 int main(int argc, char* argv[]) {
22     std::int64_t runs, lower, upper, step;
23     validate_input(argc, argv, runs, lower, upper, step);
24
25     // --- Parametros del experimento ---
26     const double PESO_MIN = 0.0;
27     const double PESO_MAX = 10.0;
28     const unsigned SEMILLA = 20260706u; // fija para reproducibilidad
29
30     std::mt19937_64 rng(SEMILLA);

```

```

31     std::uniform_real_distribution<double> peso_dist(PESO_MIN, PESO_MAX);
32
33     // Archivos de salida
34     std::ofstream csv_base("resultados/base_e4_us.csv");
35     std::ofstream csv_fw("resultados/fw_e4_us.csv");
36     auto escribir_encabezado = [](std::ofstream& f) {
37         f << "n,t_mean,t_stdev,t_Q0,t_Q1,t_Q2,t_Q3,t_Q4\n";
38     };
39     escribir_encabezado(csv_base);
40     escribir_encabezado(csv_fw);
41
42     const std::int64_t total_pruebas = ((upper - lower) / step + 1);
43     const std::int64_t total_runs = runs * total_pruebas;
44
45     std::vector<double> times(runs);
46     std::vector<double> q;
47     double mean_time, time_stdev, dev;
48     using Clock = std::chrono::high_resolution_clock;
49     using Dur = std::chrono::duration<double, std::micro>;
50
51     std::cerr << "\033[0;36mExperimento 4: grafos completos"
52               << " (n=" << lower << "..." << upper << ", step=" << step
53               << ", runs=" << runs << ")\033[0m\n\n";
54
55     std::int64_t ejecutadas = 0;
56     for (std::int64_t n = lower; n <= upper; n += step) {
57         // --- Generar grafo completo: arista entre todo par (u,v), u != v ---
58         Grafo g;
59         g.n = static_cast<int>(n);
60         for (int u = 0; u < g.n; ++u)
61             for (int v = 0; v < g.n; ++v)
62                 if (u != v)
63                     g.aristas.push_back({u, v, peso_dist(rng)});
64
65         // --- Verificacion cruzada: ambos algoritmos deben coincidir ---
66         {
67             bool cn_base = false, cn_fw = false;
68             Matriz D_base = algoritmo_base(g, &cn_base);
69             Matriz D_fw = floyd_warshall(g, &cn_fw);
70             if (cn_base != cn_fw) {
71                 std::cerr << "\n [ERROR] n=" << n
72                       << ": los algoritmos discrepan en deteccion de ciclo\n";
73                 std::exit(EXIT_FAILURE);
74             }
75         }
76
77         // ----- Medir algoritmo_base (n ejecuciones de Bellman-Ford) -----
78         mean_time = 0.0;
79         time_stdev = 0.0;
80         for (std::int64_t i = 0; i < runs; ++i) {
81             display_progress(++ejecutadas, total_runs * 2);
82             auto t0 = Clock::now();
83             // volatile evita que el compilador elimine la llamada
84             volatile auto D = algoritmo_base(g);
85             (void)D;
86             auto t1 = Clock::now();
87             times[i] = Dur(t1 - t0).count();
88             mean_time += times[i];
89         }

```

```

90     mean_time /= runs;
91     for (std::int64_t i = 0; i < runs; ++i) {
92         dev = times[i] - mean_time;
93         time_stdev += dev * dev;
94     }
95     time_stdev = std::sqrt(time_stdev / (runs - 1));
96     quartiles(times, q);
97     csv_base << n << "," << mean_time << "," << time_stdev << ","
98         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
99         << q[4] << "\n";
100
101     // ----- Medir floyd_warshall (programacion dinamica, Theta(n^3)) -----
102     mean_time = 0.0;
103     time_stdev = 0.0;
104     for (std::int64_t i = 0; i < runs; ++i) {
105         display_progress(++ejecutadas, total_runs * 2);
106         auto t0 = Clock::now();
107         volatile auto D = floyd_warshall(g);
108         (void)D;
109         auto t1 = Clock::now();
110         times[i] = Dur(t1 - t0).count();
111         mean_time += times[i];
112     }
113     mean_time /= runs;
114     for (std::int64_t i = 0; i < runs; ++i) {
115         dev = times[i] - mean_time;
116         time_stdev += dev * dev;
117     }
118     time_stdev = std::sqrt(time_stdev / (runs - 1));
119     quartiles(times, q);
120     csv_fw << n << "," << mean_time << "," << time_stdev << ","
121         << q[0] << "," << q[1] << "," << q[2] << "," << q[3] << ","
122         << q[4] << "\n";
123 }
124
125 std::cerr << "\n\n\033[1;32mExperimento 4 completado.\033[0m\n";
126 csv_base.close();
127 csv_fw.close();
128 return 0;
129 }

```

A.8. experimentos/datasets_reales.cpp

```

1 // datasets_reales.cpp
2 // Pregunta 2.3
3 // Floyd-Warshall sobre bio-SC-TS.edges y power-685-bus.mtx.
4 //
5 // Uso: g++ -std=c++17 -O2 -o datasets_reales.exe datasets_reales.cpp
6 //      ./datasets_reales.exe
7 // Salida: resultados/bio-SC-TS_resultado.csv, resultados/power-685-bus_resultado.
8         csv
9
10 #include <chrono>
11 #include <cmath>
12 #include <cstdio>
13 #include <fstream>
14 #include <iostream>

```

```

14 #include <limits>
15 #include <string>
16 #include <vector>
17
18 #include "../src/apsp.hpp"
19 #include "../src/grafo.hpp"
20 #include "../src/parser_dataset.hpp"
21
22 // ----- Utilidad -----
23 // Escribe la matriz D en formato CSV: nodo1,nodo2,distancia.
24 void escribir_csv(const Matriz& D, const std::string& ruta) {
25     std::ofstream salida(ruta);
26     if (!salida) {
27         std::fprintf(stderr, "Error: no se pudo crear %s\n", ruta.c_str());
28         return;
29     }
30     salida << "nodo1,nodo2,distancia\n";
31     const int n = static_cast<int>(D.size());
32     for (int i = 0; i < n; ++i) {
33         for (int j = 0; j < n; ++j) {
34             salida << i << "," << j << ",";
35             if (std::isinf(D[i][j]))
36                 salida << "INF";
37             else
38                 salida << D[i][j];
39             salida << "\n";
40         }
41     }
42     salida.close();
43     std::printf(" CSV escrito: %s\n", ruta.c_str());
44 }
45
46 // ----- Programa principal -----
47
48 int main() {
49     using Clock = std::chrono::high_resolution_clock;
50     std::printf("---- Pregunta 2.3: Floyd-Warshall sobre datasets reales ----\n\n");
51     ;
52
53     // -----
54     // Dataset 1: bio-SC-TS.edges
55     // Fuente: https://networkrepository.com/bio-SC-TS.php
56     // Autor: Ryan A. Rossi & Nesreen K. Ahmed (Network Repository, AAAI 2015)
57     // -----
58     {
59         std::printf("--- bio-SC-TS.edges ---\n");
60         std::printf(" Cargando...\n");
61         Grafo g = leer_edges("../bio-SC-TS.edges");
62         std::printf(" n = %d, m = %zu\n", g.n, g.aristas.size());
63
64         std::printf(" Ejecutando Floyd-Warshall...\n");
65         bool ciclo = false;
66         auto t0 = Clock::now();
67         Matriz D = floyd_warshall(g, &ciclo);
68         auto t1 = Clock::now();
69
70         auto duracion = std::chrono::duration<double>(t1 - t0).count();
71         std::printf(" Tiempo: %.6f s\n", duracion);
72         std::printf(" Ciclo negativo: %s\n", ciclo ? "SI" : "NO");

```

```

72     escribir_csv(D, "resultados/bio-SC-TS_resultado.csv");
73     std::printf("\n");
74 }
75
76
77 // -----
78 // Dataset 2: power-685-bus.mtx
79 // Fuente: https://networkrepository.com/power-685-bus.php
80 // Autor: Ryan A. Rossi & Nesreen K. Ahmed (Network Repository, AAAI 2015)
81 // -----
82 {
83     std::printf("--- power-685-bus.mtx ---\n");
84     std::printf("    Cargando...\n");
85     Grafo g = leer_mtx("../power-685-bus.mtx");
86     std::printf("    n = %d, m = %zu (incluye aristas simetricas)\n",
87                 g.n, g.aristas.size());
88
89     std::printf("    Ejecutando Floyd-Warshall...\n");
90     bool ciclo = false;
91     auto t0 = Clock::now();
92     Matriz D = floyd_warshall(g, &ciclo);
93     auto t1 = Clock::now();
94
95     auto duracion = std::chrono::duration<double>(t1 - t0).count();
96     std::printf("    Tiempo: %.6f s\n", duracion);
97     std::printf("    Ciclo negativo: %s\n", ciclo ? "SI" : "NO");
98
99     escribir_csv(D, "resultados/power-685-bus_resultado.csv");
100    std::printf("\n");
101 }
102
103 std::printf("----- Pregunta 2.3 completada ----- \n");
104 return 0;
105 }

```

Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.^a ed., MIT Press, 2009. Capítulos 24 y 25.
- [2] R. A. Rossi y N. K. Ahmed, "The Network Data Repository with Interactive Graph Analytics and Visualization," en *Proc. 29th AAAI Conf. Artificial Intelligence*, 2015. <https://networkrepository.com>
- [3] R. Williams, "Faster all-pairs shortest paths via circuit complexity," en *Proc. 46th ACM Symp. Theory of Computing (STOC)*, pp. 664–673, 2014.
- [4] U. Zwick, "All pairs shortest paths using bridging sets and rectangular matrix multiplication," *Journal of the ACM*, vol. 49, no. 3, pp. 289–317, 2002.